

# mikroflow — запуск на продакшене

Сбор NetFlow с MikroTik → PostgreSQL

2026-07-02

## Что это

---

**mikroflow** собирает с роутера MikroTik информацию о соединениях (кто и куда подключался: `src_ip → dst_ip`), обогащает её **именем устройства** (из аренд DHCP) и **доменом назначения** (reverse-DNS) и складывает в PostgreSQL.

- Сырые флоу хранятся ~14 дней, почасовые агрегаты — 6 месяцев.
- Разворачивается через Docker Compose на Linux.
- Финальный анализ — по SQL-выухе `v_connections`.

Стек из трёх сервисов: `postgres`, `collector` (приём NetFlow по UDP), `worker` (синк DHCP, reverse-DNS, агрегация, ретеншн).

---

## 1. Подготовить сервер

---

Сервер — Linux с Docker.

```
# Docker + Compose plugin (Ubuntu/Debian)
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER # перелогиниться после этого
docker compose version        # проверка
```

## 2. Доставить код на сервер

---

```
# Вариант А — через git remote (если заведёшь GitHub/GitLab)
git remote add origin <url>
git push -u origin feat/netflow-collector

# Вариант Б — просто скопировать папку на сервер
rsync -av --exclude .venv --exclude .git ./ user@server:/opt/mikroflow/
```

Код должен лежать, например, в `/opt/mikroflow`.

### 3. Настроить `.env` (ОБЯЗАТЕЛЬНО смени пароли)

---

```
cd /opt/mikroflow
cp .env.example .env
nano .env
```

Заполни:

```
MIKROFLOW_ROUTER_HOST=192.168.88.1      # IP твоего MikroTik
MIKROFLOW_ROUTER_USER=netflow            # read-only API-юзер (см. шаг 4)
MIKROFLOW_ROUTER_PASSWORD=<сильный_пароль>
MIKROFLOW_ROUTER_PORT=8728
MIKROFLOW_RAW_RETENTION_DAYS=14
MIKROFLOW_HOURLY_RETENTION_DAYS=180
```

**Важно про пароль БД.** Сейчас в `docker-compose.yml` пароль Postgres захардкожен как `mikroflow`. Для прода поменяй его в двух местах — `POSTGRES_PASSWORD` у сервиса `postgres` и в `MIKROFLOW_DB_DSN` у `collector` и `worker` (либо вынеси в `.env`). Иначе БД будет с дефолтным паролем.

### 4. Настроить MikroTik

---

В терминале роутера:

```
# экспорт потоков на сервер
/ip traffic-flow set enabled=yes interfaces=all
/ip traffic-flow target add address=<IP_СЕРВЕРА>:2055 version=9

# отдельный read-only пользователь для чтения аренд DHCP через API
/user group add name=netflow-ro policy=api,read,test
/user add name=netflow group=netflow-ro password=<тот_же_пароль_что_в_.env>
/ip service enable api
```

### 5. Запустить

---

```
cd /opt/mikroflow
docker compose up -d --build
docker compose ps          # все три сервиса Up (postgres healthy)
docker compose logs -f collector worker
```

## 6. Проверить, что данные идут

---

```
# сырые флоу должны появиться в течение минуты-двух
docker compose exec postgres psql -U mikroflow \
  -c "SELECT count(*) FROM flows_raw;"

# аренды DHCP подтянулись (даёт имена устройств)
docker compose exec postgres psql -U mikroflow \
  -c "SELECT count(*) FROM dhcp_leases WHERE valid_to IS NULL;"

# агрегаты появятся после первого полного часа
docker compose exec postgres psql -U mikroflow \
  -c "SELECT * FROM v_connections ORDER BY bytes DESC LIMIT 20;"
```

Пример аналитического запроса:

```
SELECT hour, device_name, src_ip, dst_domain, dst_ip, dst_port,
       bytes, flow_count
FROM v_connections
WHERE hour >= now() - interval '7 days'
ORDER BY bytes DESC
LIMIT 100;
```

## 7. Что учесть для прода

---

- **Файрвол сервера:** открой входящий **UDP 2055** только с IP роутера. API MikroTik (8728) — исходящий с сервера.
- **Данные переживают перезапуск:** Postgres в volume `pgdata`, стек с `restart: unless-stopped`. Партиции создаются и чистятся воркером автоматически (`raw` — 14 дней, `hourly` — 180 дней).
- **Бэкап:** `docker compose exec postgres pg_dump -U mikroflow mikroflow > backup.sql`.
- **Тюнинг под 100+ устройств:** при пропусках подними `MIKROFLOW_RECV_BUFFER_BYTES` и `MIKROFLOW_BATCH_SIZE` в `.env`.
- **Reverse-DNS:** домены CDN могут не резолвиться — это принятое ограничение выбранного лёгкого пути (NetFlow без зеркалирования трафика).